

Observability — A Readable Companion to the Elixir Docs

Chapter 13 · checked against Elixir 1.20 · examples run on Elixir 1.14 · Sep 26, 2026

1. Why observability in Elixir is mostly the BEAM's

Elixir runs on a runtime you can inspect from the inside: list processes, read their state, trace calls, measure schedulers, all on a live node (Erlang book, chapter 9). Elixir adds two things on top that most applications use every day:

- **Logger**, Elixir's interface to Erlang's `:logger`, with metadata and macros that skip work when a level is disabled.
- `:telemetry`, the ecosystem-wide library through which Phoenix, Ecto, Oban, Broadway and others publish events: request durations, query times, job results. Metrics, dashboards (Phoenix LiveDashboard) and tracing integrations all subscribe to these events.

Both are easy to use in ways that cost performance or lose exactly the information you need during an incident.

2. The building blocks

Logger

Feature	Meaning
<code>Logger.debug/info/warning/error(...)</code>	Macros; the message is evaluated only if the level is enabled
<code>Logger.debug(fn -> ... end)</code>	Explicitly lazy message
<code>Logger.metadata(request_id: id)</code>	Attach key-value context to every later log line in this process
Structured reports	Log a map or keyword list; formatters and handlers keep the fields
<code>compile_time_purge_matching</code>	Remove chosen log calls at compile time
Erlang <code>:logger</code> underneath	Handlers, filters and overload protection (Erlang book, chapter 9)

Telemetry

Function	Meaning
<code>:telemetry.execute(event, measurements, metadata)</code>	Emit an event, e.g. <code>[:my_app, :checkout, :stop]</code>
<code>:telemetry.span(event, meta, fun)</code>	Emit <code>:start</code> and <code>:stop</code> (or <code>:exception</code>) events with durations around <code>fun</code>
<code>:telemetry.attach(id, event, handler, config)</code>	Subscribe a handler function to an event

The telemetry docs stress one point: “The `handle_event` callback of each handler is invoked synchronously on each `telemetry:execute` call.” Handlers run inside the process that emits the event, in the middle of its work.

3. How it actually works

Logger macros skip work, but only for their own arguments

Because `Logger.debug/1` is a macro, `Logger.debug("state: #{inspect(state)}")` checks the level before building the string. With debug disabled, the interpolation never runs (verified). But if you build the message in a variable first, it is built regardless of the level (verified).

Metadata lives in the process

`Logger.metadata/1` stores context in the current process. New processes start with none: inside a `Task.async` spawned from a request, `Logger.metadata()` is empty (verified), so the task’s log lines have no `request_id`.

Handlers can detach themselves

If a telemetry handler raises, telemetry detaches it (so the failure can’t repeat on every event) and logs a warning. The metric it was feeding silently stops updating until the application restarts, while the rest of the system works normally.

Handlers cost the emitter

Everything a handler does, like formatting, writing to a socket, or calling a `GenServer`, adds latency to the request, query or job that emitted the event. A slow metrics backend becomes slow requests.

4. Plausible but wrong

4.1 Building the log message before logging

```
msg = "processing order #{inspect(order, limit: :infinity)}"  
Logger.debug(msg)
```

The message is built before `Logger.debug/1` can check the level, so with debug disabled in production the full `inspect` still runs on every order (verified). Fix: put the expression inside the call (`Logger.debug("processing order #{order.id}")`) or pass a function. Log identifiers, not whole structures.

4.2 Losing request context in spawned work

```
def create(conn, params) do
  Logger.metadata(request_id: conn.assigns.request_id)
  Task.async(fn -> Billing.charge(params) end) |> Task.await()
end
```

Log lines from `Billing.charge/1` run in the task, which has no metadata (verified), so they can't be tied to the request. That is exactly what you need when a charge fails. Fix: capture and set it in the new process:

```
md = Logger.metadata()
Task.async(fn -> Logger.metadata(md); Billing.charge(params) end)
```

4.3 A telemetry handler that does I/O

```
:telemetry.attach("audit", [:my_app, :order, :stop], fn _e, m, meta, _ ->
  HTTPClient.post!(@audit_url, %{order: meta.order_id, ms: m.duration})
end, nil)
```

The HTTP call runs inside every order request, synchronously. When the audit service slows down, checkout slows down, and when it fails, the handler raises and is detached (4.4). Fix: keep handlers cheap. Update a counter or ETS table, or send a message to a process that batches and ships the data.

4.4 A handler that raises once and disappears

```
def handle_event([:repo, :query], %{total_time: t}, %{source: source}, _) do
  Metrics.observe("db.query", t, table: String.to_existing_atom(source))
end
```

The first query without a `:source` (a raw SQL query, a migration) crashes the handler. Telemetry detaches it and logs one warning, and database metrics stop for the rest of the node's life while dashboards show a quiet, healthy flat line. Fix: match defensively on metadata in handlers (a final clause that ignores unexpected shapes), alert on missing metrics, and check logs for detach warnings.

4.5 Anonymous functions as handlers

```
:telemetry.attach("req", [:phoenix, :endpoint, :stop], fn _, m, _, _ ->
  Metrics.observe("http", m.duration)
end, nil)
```

It works, but the telemetry docs recommend function captures (`&MyApp.Telemetry.handle_event/4`) for performance: they are cheaper to call on every event, and appear by name in traces and warnings. An anonymous function defined in `iex` or a script also doesn't survive code reloading the way a module function does. Fix: define handlers as public functions in a module and attach with `&Module.fun/4`.

4.6 Logging instead of measuring

```
Logger.info("checkout took #{duration} ms")
```

Latency hidden in log lines can't be graphed, aggregated or alerted on without parsing text, and under load logger overload protection may drop some lines (Erlang book, chapter 9, section 4.6). Fix: emit a telemetry event (`:telemetry.execute([:my_app, :checkout, :stop], %{duration: d}, meta)`), feed it to metrics, and keep logs for events you need to read.

4.7 Debug tooling left on in production

```
:sys.trace(MyApp.OrderServer, true)    # "just for a minute"
```

`:sys.trace/2` prints every event of that process, and `dbg`-style tracing on busy functions can flood a node (Erlang book, chapter 9, section 4.3). Left on, it fills logs and slows the process. Fix: use time-boxed or rate-limited tools (`recon_trace`), and turn tracing off explicitly (`:sys.trace(pid, false)`, `:sys.no_debug/1`), as the GenServer docs also advise.

5. Review checklist and sources

- ☐ Log messages are built inside the Logger call (or a function), never beforehand
- ☐ Logs carry identifiers, not whole structs or payloads
- ☐ Request metadata is copied into tasks and other spawned processes
- ☐ Telemetry handlers are cheap, never do I/O, and hand work off to other processes
- ☐ Handlers tolerate unexpected metadata; detach warnings are monitored
- ☐ Handlers are attached as `&Module.fun/4` captures
- ☐ Durations and counts are telemetry metrics, not log lines
- ☐ Tracing in production is time-boxed and switched off explicitly

Sources

- Logger: <https://hexdocs.pm/logger/Logger.html>
- telemetry: <https://hexdocs.pm/telemetry/readme.html>
- GenServer (debugging with `:sys`): <https://hexdocs.pm/elixir/GenServer.html>
- Erlang book, chapter 9 (Observability): <https://github.com/ayarodionov/Learning-Erlang-from-Claude>